

RAG Metrics & Chunking Evaluation Report

Performance Analysis of Recursive vs. Semantic Strategies across Infrastructure Configurations

Dataset: engineer_questions.csv (15 evaluation items)

Date: September 2026

Domain: Halcyon Turbine HT-400 Maintenance Docs

Scope: Generator & Retrieval Metrics Analysis

1. Executive Summary

This evaluation report provides a detailed benchmark of Retrieval-Augmented Generation (RAG) system performance using two distinct chunking strategies (**Recursive** and **Semantic**) tested under two separate deployment environments: local open-source models (phi3:mini) and cloud-hosted enterprise LLMs via Portkey (Azure OpenAI GPT-5.4).

Key Benchmark Finding

The choice of **LLM Judge & Generator** (Local phi3:mini vs. Enterprise GPT-5.4) proved to be the single most critical factor in evaluation stability. Local small language models suffered a **80% execution failure rate** due to structured JSON output parsing errors, whereas Portkey-managed enterprise endpoints achieved a **100% completion rate** with high output fidelity.

2. Individual Run Results & Metrics Summary

Below is the aggregate metric comparison across all four evaluated JSON configuration runs. Each run evaluated 15 technical engineering queries regarding field maintenance procedures.

FILE NAME	CHUNKING STRATEGY	JUDGE & GENERATOR MODEL	EVAL STATUS	ANSWER RELEVANCY	FAITHFULNESS	CONTEXT RELEVANCY
recursive.json	Recursive (800/100)	phi3:mini (Local)	3/15 (20%)	0.3786	0.7386	0.3703
semantic.json	Semantic (p95)	phi3:mini (Local)	3/15 (20%)	0.5582	0.3000	0.4820
recursive-portkey.json	Recursive (800/100)	Azure GPT-5.4 (Portkey)	15/15 (100%)	0.8439	0.9717	0.2678
semantic-portkey.json	Semantic (p85)	Azure GPT-5.4 (Portkey)	15/15 (100%)	0.8378	0.9773	0.2695

Detailed Breakdown per Configuration

1. Local Baseline: recursive.json

- **Chunking Settings:** Recursive strategy, chunk size 800 tokens, overlap 100 tokens, embedding model all-MiniLM-L6-v2.
- **Performance:** 12 out of 15 test items failed evaluation due to JSON output schema enforcement errors in phi3:mini.
- **Metrics (3 valid items):** Relevancy was low (0.3786), while Faithfulness remained moderate (0.7386). Retrieval Contextual Relevancy averaged 0.3703.

2. Local Baseline: semantic.json

- **Chunking Settings:** Semantic strategy (percentile threshold 95.0), embedding model nomic-embed-text.
- **Performance:** Experienced identical failure rates (12/15 failed) due to evaluator model limitations.
- **Metrics (3 valid items):** Answer Relevancy reached 0.5582, but Faithfulness dropped severely to 0.3000. Contextual Relevancy was highest among local runs at 0.4820.

3. Enterprise Pipeline: recursive-portkey.json

- **Chunking Settings:** Recursive strategy (800 size, 100 overlap), embedding model nomic-embed-text.
- **Performance:** Flawless execution with 15/15 queries successfully evaluated using Azure OpenAI GPT-5.4 via Portkey gateway.
- **Metrics:** High Answer Relevancy (0.8439) and near-perfect Faithfulness (0.9717). Contextual Relevancy logged at 0.2678 (Top-K = 4).

4. Enterprise Pipeline: semantic-portkey.json

- **Chunking Settings:** Semantic strategy (percentile threshold 85.0), embedding model nomic-embed-text.
- **Performance:** 100% completion (15/15 evaluated).
- **Metrics:** Answer Relevancy of 0.8378, top-tier Faithfulness of 0.9773, and Contextual Relevancy of 0.2695.

3. Comparative Analysis & Conclusions

A rigorous comparison of the results yields critical insights into both the infrastructure requirements for RAG evaluation and the specific impact of chunking strategies.

Local Models vs. Portkey Pipeline

Local (Phi-3 Mini): Unsuitable as an automated RAG judge. Failed 80% of evaluation queries due to invalid JSON syntax formatting. The small sample size (n=3) yields statistically unreliable aggregate scores.

Portkey (Azure GPT-5.4): Delivered 100% reliability, zero schema errors, and robust quality scores. Enterprise gateways with strong frontier models are essential for automated RAG benchmark pipelines.

Recursive vs. Semantic Chunking

Recursive Chunking (Portkey): Achieved slightly higher Answer Relevancy (0.8439 vs 0.8378). It maintains predictable token boundaries and is computationally light.

Semantic Chunking (Portkey): Achieved slightly superior Faithfulness (0.9773 vs 0.9717) and equivalent retrieval precision. Preserves natural topic transitions at section boundaries.

Key Conclusions & Strategic Recommendations

- 1. Chunking Strategy Equivalence in Structured Technical Docs:** Under robust generation (GPT-5.4), Recursive and Semantic chunking perform almost identically across all core metrics (Answer Relevancy: 84.4% vs 83.8%; Faithfulness: 97.2% vs 97.7%). For structured technical manuals with distinct section headers, fixed Recursive chunking is recommended due to lower processing overhead.
- 2. Retrieval Context Noise (Contextual Relevancy ~26.8%):** In both Portkey runs, Contextual Relevancy remained relatively low (~0.27). This indicates that retrieving a fixed `top_k = 4` passages inserts extra context that, while helpful for context, contains non-essential sentences. Introducing a re-ranker or metadata filter could further optimize prompt context space.
- 3. Evaluation Infrastructure Maturity:** Local small language models cannot reliably act as evaluators without constrained JSON decoding logic. Standardizing on managed enterprise gateways (Portkey) with frontier LLMs guarantees repeatable evaluation runs.